Python Data Class Builders & Schema Design

A Comprehensive Handbook for Modern Data Modeling in Python

Reference Resource: Fluent Python & Modern Standard Library | Week 1 Core Curriculum

---

Table of Contents

- **Section 1: The Problem with Manual Classes**
  Boilerplate overload in __init__, __repr__, and __eq__
- **Section 2: Classic vs Typed Named Tuples**
  collections.namedtuple vs typing.NamedTuple
- **Section 3: The @dataclass Decorator**
  Automatic method synthesis and standard usage
- **Section 4: Field Customization with field()**
  default_factory, compare, repr, and mutable defaults
- **Section 5: Post-Initialization & Invariants**
  __post_init__ validation and derived attributes
- **Section 6: Immutability & Thread Safety**
  frozen=True, hashability, and dictionary keys
- **Section 7: Type Annotations & Special Variables**
  ClassVar vs InitVar and type checking semantics
- **Section 8: Bridge to Pydantic V2**
  Transitioning from @dataclass to Pydantic BaseModel for APIs

Primary Goal for Week 1: Master building clean, self-validating, typed Python data models for calendar events, time slots, and tasks without writing repetitive boilerplate code.

1. The Problem with Manual Classes

When writing data-centric applications—such as a calendar scheduler that handles events, time windows, and task items—most classes exist primarily to hold a collection of attributes. Writing plain Python classes requires extensive boilerplate:

```
class CalendarEvent:
    def __init__(self, id: str, summary: str, start: str, end: str):
        self.id = id
        self.summary = summary
        self.start = start
        self.end = end

    def __repr__(self):
        return f"CalendarEvent(id={self.id!r}, summary={self.summary!r})"

    def __eq__(self, other):
        if not isinstance(other, CalendarEvent):
            return False
        return (self.id, self.summary, self.start, self.end) == (other.id, other.summary, other.start, other.end)
```

Without explicit implementations of __repr__ and __eq__, instance printing returns unhelpful object IDs (e.g. <CalendarEvent object at 0x...>) and equality checks fall back to object identity (is) rather than value equality.

2. Classic vs Typed Named Tuples

Python provides two built-in tuple-based class builders: collections.namedtuple and typing.NamedTuple.

```
from collections import namedtuple
from typing import NamedTuple
from datetime import datetime

# 1. Classic namedtuple (Untyped runtime builder)
Slot = namedtuple('Slot', ['start', 'end', 'duration'])

# 2. Modern typing.NamedTuple (Type-annotated class syntax)
class TimeSlot(NamedTuple):
    start: datetime
    end: datetime
    duration_minutes: int = 30
```

Key Characteristics of NamedTuples:

• Instances are immutable and memory-efficient (they have tuple memory overhead).

• Fields can be accessed by name (slot.start) or index (slot[0]).

• They support tuple unpacking (start, end, duration = slot).

Limitations: Because they inherit from tuple, accidental comparisons with plain tuples can cause unexpected equality matches. They also cannot be mutable.

3. The @dataclass Decorator

Introduced in Python 3.7 (PEP 557), @dataclass is a class decorator that inspects variable annotations (PEP 526) to automatically generate __init__, __repr__, __eq__, and other special methods.

```
from dataclasses import dataclass
from datetime import datetime

@dataclass
class CalendarEvent:
    id: str
    summary: str
    start: datetime
    end: datetime
    is_all_day: bool = False

# Usage:
ev1 = CalendarEvent("evt_1", "Sprint Review", datetime(2026, 10, 24, 10), datetime(2026, 10, 24, 11))
print(ev1)
# Output: CalendarEvent(id='evt_1', summary='Sprint Review', start=datetime(...), end=datetime(...), is_all_day=Fal
```

Unlike NamedTuples, instances of @dataclass are real Python class instances (not tuple subclasses). They are mutable by default and do not support index-based access.

4. Field Customization with field()

In Python, mutable default arguments (like tags: list = []) are a notorious source of bugs. The dataclasses.field() function prevents this and allows granular control over each attribute:

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class TaskDraft:
    title: str
    importance: int
    urgency: int
    tags: List[str] = field(default_factory=list)  # Avoids shared mutable list
    temp_score: float = field(init=False, repr=False, compare=False)  # Derived field

    def __post_init__(self):
        self.temp_score = 0.5 * self.importance + 0.5 * self.urgency
```

Common field() Options:

• default_factory: Zero-argument callable used to generate defaults for mutable types (lists, dicts, sets).
• init=False: Excludes the field from the generated __init__ parameter list.
• repr=False: Omits the field from the string representation.
• compare=False: Ignores the field when checking equality (==) or ordering.

5. Post-Initialization & Validation (__post_init__)

When the generated __init__ finishes assigning attributes, it calls __post_init__(self) if defined. This is where you enforce business invariants and validate constraints:

```
from dataclasses import dataclass
from datetime import datetime

@dataclass
class TimeInterval:
    start: datetime
    end: datetime

    def __post_init__(self):
        if self.end <= self.start:
            raise ValueError(f"End time ({self.end}) must be strictly after start time ({self.start})")

    @property
    def duration_minutes(self) -> float:
        return (self.end - self.start).total_seconds() / 60.0
```

6. Immutability & Hashability (frozen=True)

To use data class instances as keys in dictionaries or elements in sets, the objects must be hashable. Setting frozen=True makes the instances read-only and automatically generates a __hash__ method based on the fields:

```
@dataclass(frozen=True)
class EventKey:
    calendar_id: str
    event_id: str

key1 = EventKey("primary", "evt_101")
cache = {key1: "Sprint Planning"}
# key1.event_id = "evt_102"  # Raises dataclasses.FrozenInstanceError!
```

7. Special Annotations: ClassVar & InitVar

Because @dataclass treats all type-annotated fields as instance fields, you must use special typing wrappers for class attributes and init-only parameters:

```
from dataclasses import dataclass, InitVar
from typing import ClassVar

@dataclass
class ScheduleConfig:
    min_buffer_minutes: int
    working_hours_start: int = 9
    working_hours_end: int = 18

    # ClassVar: Shared across all instances, ignored by __init__
    DEFAULT_TIMEZONE: ClassVar[str] = "Asia/Kolkata"

    # InitVar: Passed to __init__ and __post_init__, but NOT saved as an attribute
    apply_rounding: InitVar[bool] = True

    def __post_init__(self, apply_rounding: bool):
        if apply_rounding and self.min_buffer_minutes % 5 != 0:
            self.min_buffer_minutes = ((self.min_buffer_minutes // 5) + 1) * 5
```

8. The Bridge to Pydantic V2 (For LLMs & APIs)

While standard @dataclass is ideal for internal algorithm logic, Pydantic BaseModel is essential when receiving untrusted data from the Google Calendar API or LLM completions because Pydantic performs strict runtime type coercion and schema generation:

```
from pydantic import BaseModel, Field, field_validator
from datetime import datetime
from typing import Optional

class ExtractedTask(BaseModel):
    title: str = Field(description="Cleaned title of the task")
    importance: int = Field(ge=1, le=5, description="Scale 1 to 5")
    urgency: int = Field(ge=1, le=5, description="Scale 1 to 5")
    estimated_duration: Optional[int] = Field(default=None, description="Duration in minutes")
    is_locked: bool = Field(default=False)

    @field_validator("importance", "urgency")
    @classmethod
    def validate_bounds(cls, v: int) -> int:
        if not (1 <= v <= 5):
            raise ValueError("Score must be between 1 and 5")
        return v
```

Summary Checklist for Week 1:

1. Use @dataclass(frozen=True) for immutable interval structures (like TimeSlot).
2. Use field(default_factory=...) whenever an attribute defaults to a list or dict.
3. Use Pydantic BaseModel for schema parsing at API and LLM boundaries.